Beans that think

a gallery of beancraft programs, drawn as machines

beancraft has one kind of value, a bin of beans, and three instructions: `give R` puts a bean in bin `R`; `take R z` takes one out, or jumps to `z` if the bin is empty; and `stop`. That is a counter machine (a Minsky register machine), and everything below, from long division to a game player that learns, is built from those three moves.

Each program comes with a short description and a **flow graph** of the program as written. The graphs use the same pictures as `beancraft --emit-dot`:



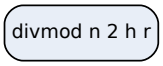
give R: put a bean in bin `R`, then carry on down.



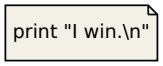
take R: if `R` has a bean, take it and carry on *down*; if it is empty, leave by the *side* arrow, the one with the small open circle at its tail.



stop.



A call of a `func` (a subroutine, inlined where it is called). Dashed side arrows are its *label arguments*, the places it can jump to instead of returning: `sub A B R` jumps to `under` when `B > A`, `readnum` to `eof` at end of input. The funcs a program defines are drawn after it; the standard library's are at the back.



A run of `putc` calls, shown as the text it prints.

→ name — A jump to a label outside the picture (only in excerpts).

Text beside a node is its label. Unconditional jumps (`take z X X` on the never-filled bin `z`) are drawn as plain arrows. These graphs are the program *as written*; the machine that actually runs has every call expanded, which is the "counter-machine instructions" count given with each program.

Run them with `./beancraft examples/NAME.bc [REG=VALUE ...]`. Most are much faster with `-O`, which recognises the standard loops (zeroing, moving, multiplying, dividing) and does each in one big-number step; that is what lets `pi.bc` print hundreds of digits and `nim.bc` play with heaps of 10^{21} beans.

Arithmetic

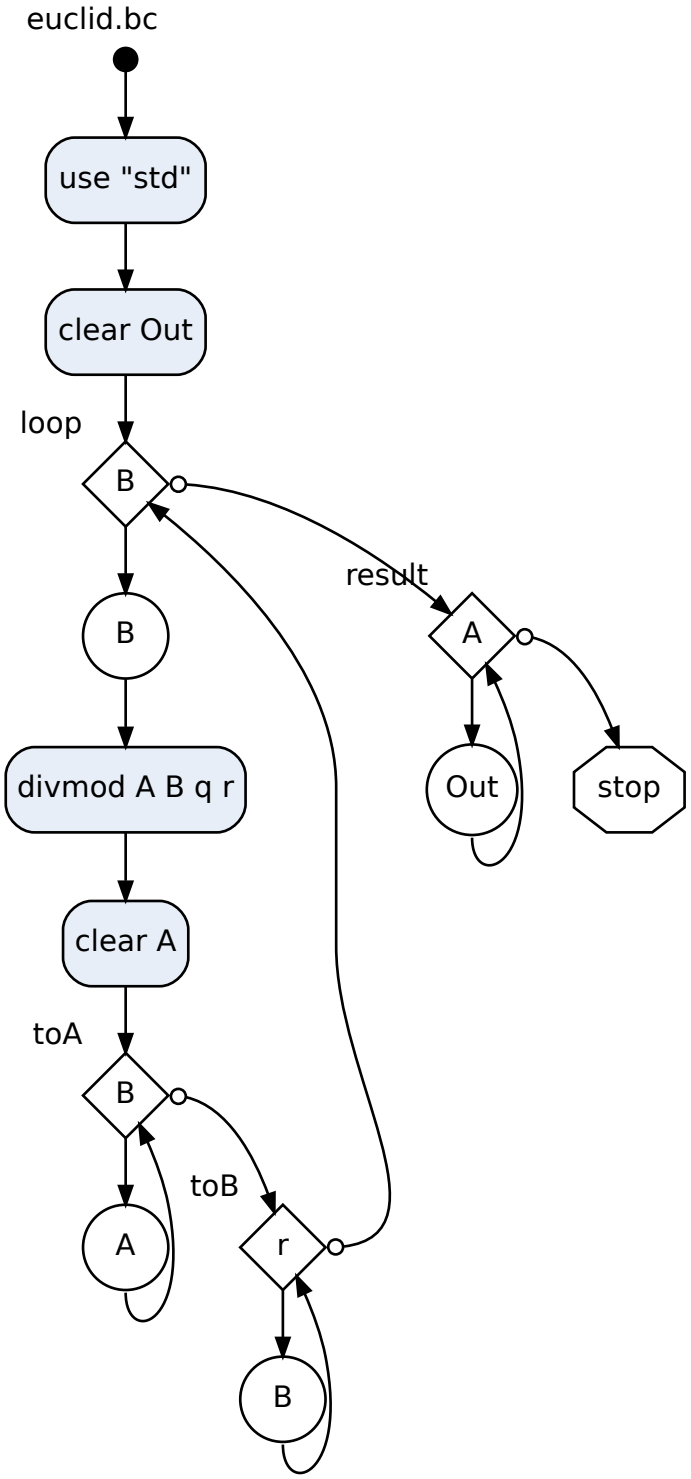
`euclid.bc`: greatest common divisor

`examples/euclid.bc` — 12 source lines · 58 counter-machine instructions · 11 bins

Euclid's algorithm with remainders: while $B > 0$, replace (A, B) by $(B, A \bmod B)$; then A is the gcd. The older `gcd.bc` does the same job by repeated subtraction, which takes about $\max(A, B)/\gcd$ rounds; this takes $O(\log \min(A, B))$ rounds, and each `divmod` is a single folded division under `-O`.

Note how little is left once `divmod` exists: the loop is a test of `B`, a call, and two *transfers* (the `take X ... / give Y` pairs that pour one bin into another).

```
./beancraft -O examples/euclid.bc A=1071 B=462                    Out = 21
```

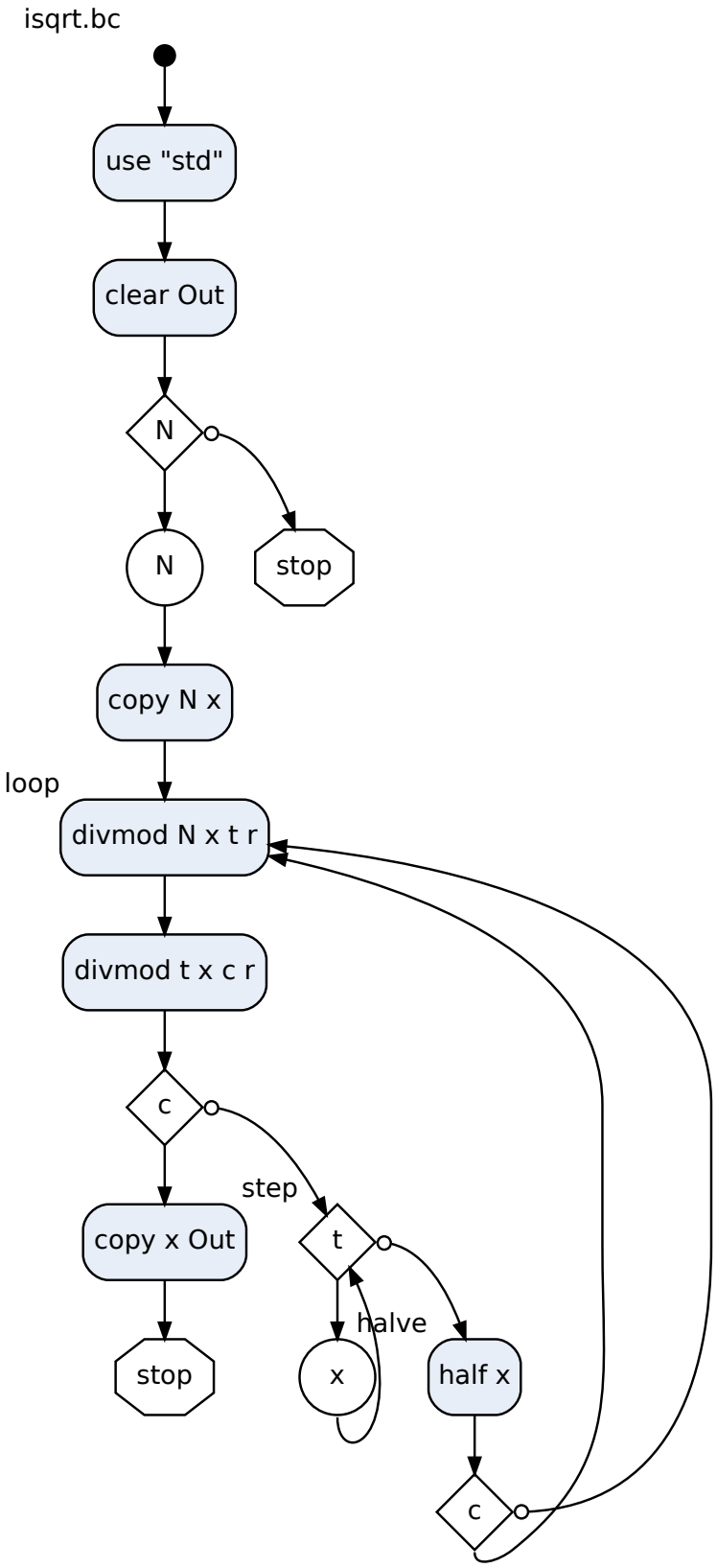


`isqrt.bc`: integer square root

`examples/isqrt.bc` — 14 source lines · 121 counter-machine instructions · 20 bins

Out $= \lfloor \sqrt{N} \rfloor$ by Newton's method on integers. Start from $x = N$ and repeat $x \leftarrow (x + N \div x) \div 2$; it falls steadily until it lands on the answer. The new x is smaller exactly when $t = N \div x < x$, and the program tests that by dividing again: $t \div x = 0$. That trick, *compare by dividing*, turns up in several programs below, because a division is one step under `-O` while counting two bins down side by side is not.

```
./beancraft -O examples/isqrt.bc N=200000000000000000000000000000000000000
                                                    Out = 44721359549995793928 (that's sqrt 2 to 20 digits)
```

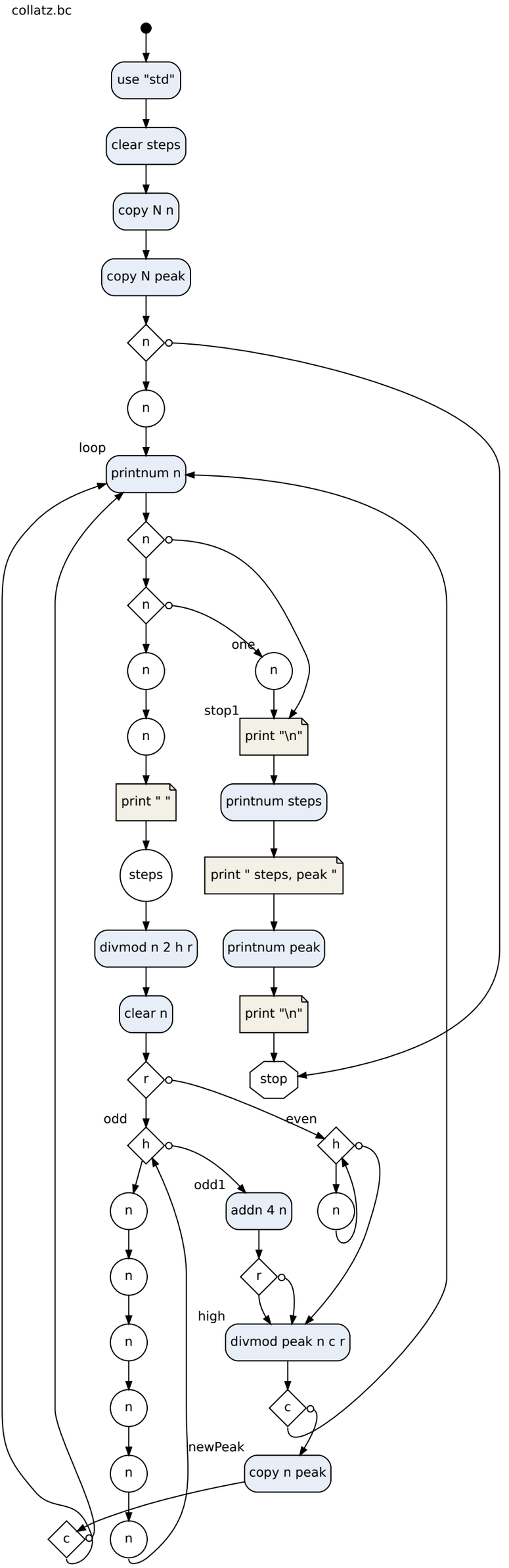


`collatz.bc`: the $3n + 1$ problem

`examples/collatz.bc` — 53 source lines · 2122 counter-machine instructions · 110 bins

Print the Collatz sequence from N (halve it if even, $3n + 1$ if odd) until it reaches 1, then the number of steps and the highest point. `divmod n 2 h r` splits $n = 2h + r$ at once, so the odd case can go straight to $6h + 4$: six `give n`s per bean of `h`, plus four. Whether every N gets to 1 is the Collatz conjecture, open since 1937.

```
./beancraft -O examples/collatz.bc N=27
27 82 41 124 62 31 94 47 ... 4 2 1
111 steps, peak 9232
```


collatz.bc
use "std"
clear steps
copy N n
copy N peak
n
n
loop
printnum n
n
n
one
n
n
stop1
print "\n"
n
print " "
printnum steps
steps
print " steps, peak "
divmod n 2 h r
printnum peak
clear n
print "\n"
r
stop
odd
even
h
h
odd1
n
addn 4 n
n
n
r
high
n
divmod peak n c r
n
c
n
newPeak
n
copy n peak
c
n

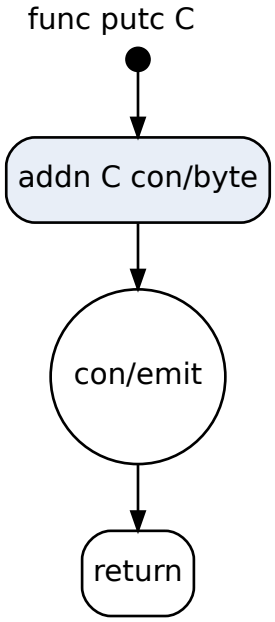


Figure 3: `func putc C`: print the character with code `C`. Every program that talks uses this.

Digits of e and π

A *spigot* produces digits one at a time and never looks at them again. The textbook spigots keep an array of digits, and a counter machine has no arrays; but Gibbons' *unbounded* spigot (2006) keeps only a few numbers, each growing without limit, and beancraft's bins are exactly that. Both programs keep the part of the series consumed so far as one fraction $f(y) = (qy + b)/t$, where $y \in [0, 1]$ stands for the rest of the series, and alternate two moves:

- **consume** the next term, which multiplies into q, b, t;
- **produce** a digit $d = b \div t$, as soon as it is certain: when $f(0)$ and $f(1)$ agree on it, i.e. $(b \bmod t) + q < t$. Print it, and shift: $b \leftarrow 10(b \bmod t)$, $q \leftarrow 10q$.

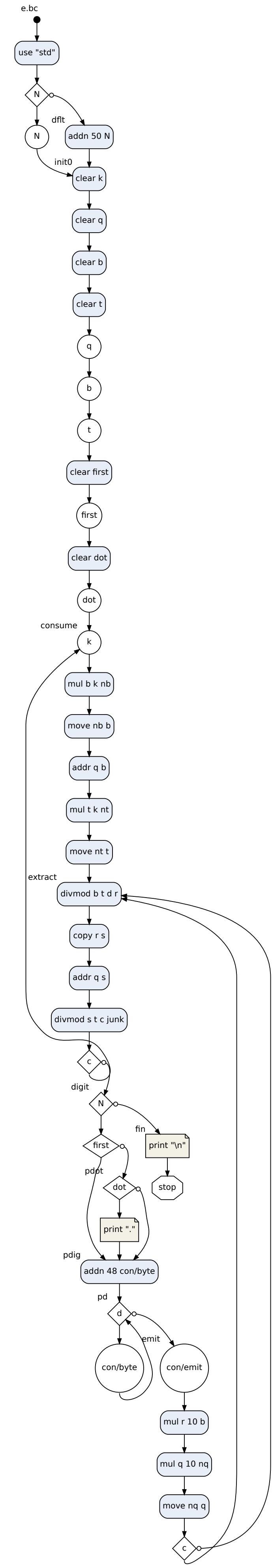
The digit test is the compare-by-dividing trick again. The fraction is set up (with y measured from the bottom of its range) so that every quantity stays non-negative, since a bin can't hold a negative number.

`e.bc`

`examples/e.bc` — 48 source lines · 459 counter-machine instructions · 49 bins

$e = 1 + \frac{1}{1}\big(1 + \frac{1}{2}\big(1 + \frac{1}{3}(1 + \cdots)\big)\big)$, so term k is the map $y \mapsto (1 + y)/k$, and consuming it is $b \leftarrow bk + q$, $t \leftarrow tk$.

```
./beancraft -O examples/e.bc N=60
2.718281828459045235360287471352662497757247093699959574966 96
```


e.bc
use "std"
N
dflt
N
addn 50 N
init0
clear k
clear q
clear b
clear t
q
b
t
clear first
first
clear dot
dot
consume
k
mul b k nb
move nb b
addr q b
mul t k nt
move nt t
extract
divmod b t d r
copy r s
addr q s
divmod s t c junk
c
digit
N
fin
print "\n"
stop
first
pdot
dot
print "."
pdig
addn 48 con/byte
pd
d
emit
con/byte
con/emit
mul r 10 b
mul q 10 nq
move nq q
c

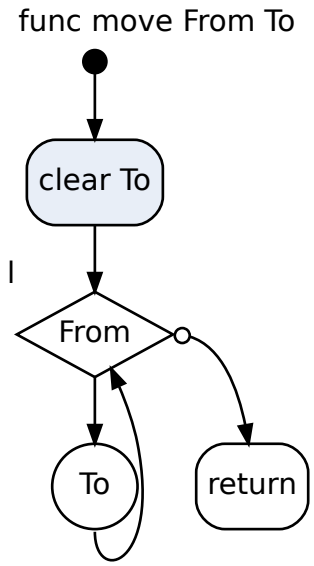


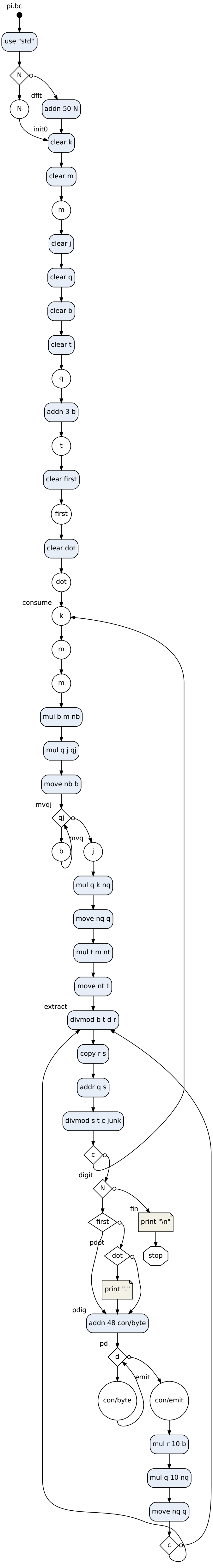
Figure 4: `func move From To`: pour `From` into `To` (`To` cleared first).

pi.bc

`examples/pi.bc` — 58 source lines · 523 counter-machine instructions · 58 bins

Euler's series $\pi = 2 + \frac{1}{3}\big(2 + \frac{2}{5}\big(2 + \frac{3}{7}(2 + \cdots)\big)\big)$; with the tail written as $3 + y$, term k becomes $b \leftarrow b(2k+1) + q(k-1)$, $q \leftarrow qk$, $t \leftarrow t(2k+1)$. It is the same machine as `e.bc`, fed a different series. Without `-O` even `3.1` takes twenty seconds; with it, 400 digits take about one.

```
./beancraft -O examples/pi.bc N=60
3.14159265358979323846264338327950288419716939937510582097494
```


pi.bc
use "std"
N
dflt
N
addn 50 N
init0
clear k
clear m
m
clear j
clear q
clear b
clear t
q
addn 3 b
t
clear first
first
clear dot
dot
consume
k
m
m
mul b m nb
mul q j qj
move nb b
mvqj
qj
mvq
b
j
mul q k nq
move nq q
mul t m nt
move nt t
extract
divmod b t d r
copy r s
addr q s
divmod s t c junk
c
digit
N
fin
print "\n"
stop
first
pdot
dot
print "."
pdig
addn 48 con/byte
pd
d
emit
con/byte
con/emit
mul r 10 b
mul q 10 nq
move nq q
c

Games against the machine

Can a few bins of beans play a game well? Five ways of saying yes.

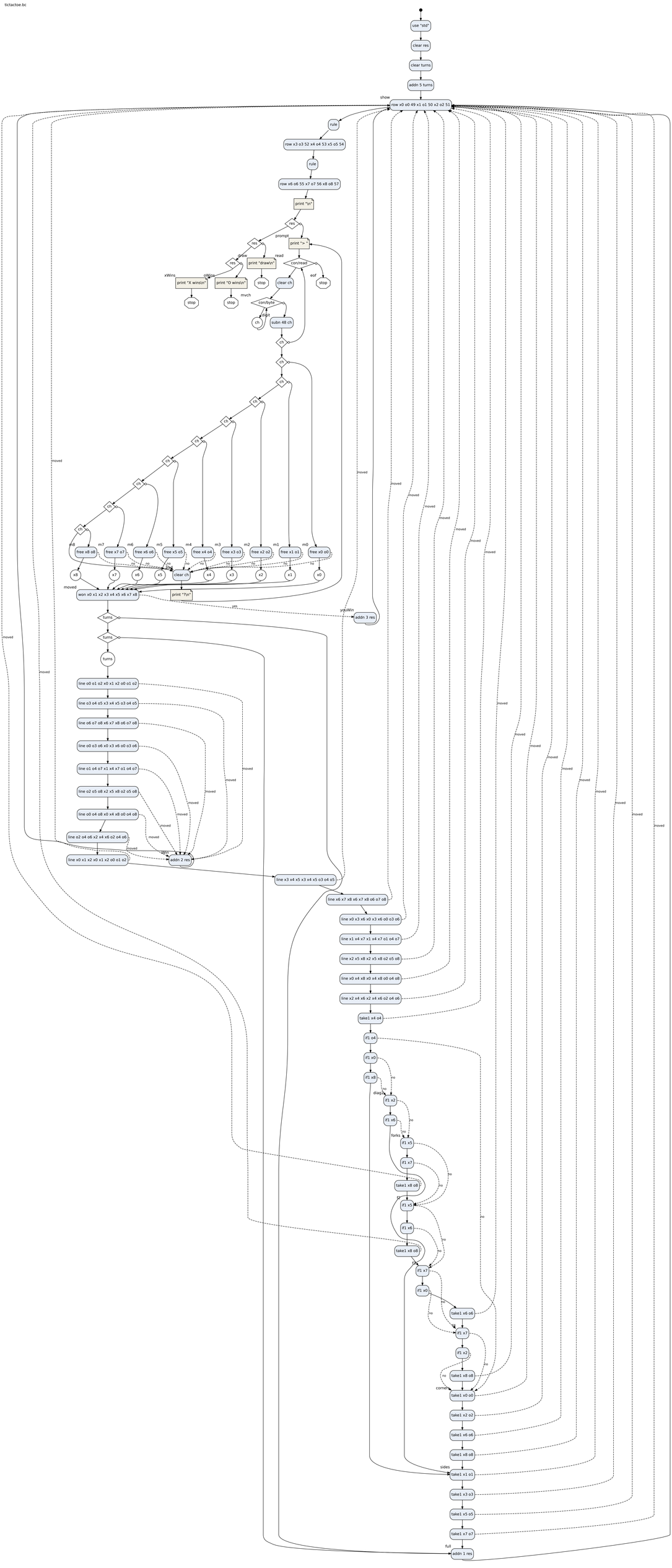
`tictactoe.bc`: rules that never lose

`examples/tictactoe.bc` — 215 source lines · 8275 counter-machine instructions · 216 bins

You are X and move first; the machine is O. Each square is two bins, `x`i and `o`i. The machine plays the first rule that applies: win if it can; block your line; take the centre; if you hold two opposite corners, take a side (a corner loses to a fork); four specific "fork-stopper" replies; else a corner, else a side. The rules were found by trial in Python and then checked by playing the actual program against every possible sequence of X moves: 489 games, 348 won by O, 141 drawn, none lost.

`func line` tries to complete one line in each of its three orientations; the long column of `line` calls in the middle of the graph is the win check and then the block check, eight lines each.

```
./beancraft examples/tictactoe.bc                    type a square 1–9
```


use "std"
clear res
clear turns
addn 5 turns
show
row x0 o0 49 x1 o1 50 x2 o2 51
rule
row x3 o3 52 x4 o4 53 x5 o5 54
rule
row x6 o6 55 x7 o7 56 x8 o8 57
print "\n"
res
prompt
print ">"
res
grow
print "draw\n"
read
con/read
xWins
print "X wins\n"
oWins
print "O wins\n"
stop
clear ch
eof
stop
stop
stop
rnvch
con/byte
digit
ch
subn 48 ch
ch
ch
ch
ch
ch
ch
ch
ch
ch
ch
m8
free x8 o8
m7
free x7 o7
m6
free x6 o6
m5
free x5 o5
m4
free x4 o4
m3
free x3 o3
m2
free x2 o2
m1
free x1 o1
m0
free x0 o0
x8
x7
x6
x5
clear ch
x4
x3
x2
x1
x0
moved
won x0 x1 x2 x3 x4 x5 x6 x7 x8
print "/\n"
yes
youWin
addn 3 res
turns
turns
turns
line o0 o1 o2 x0 x1 x2 o0 o1 o2
line o3 o4 o5 x3 x4 x5 o3 o4 o5
line o6 o7 o8 x6 x7 x8 o6 o7 o8
line o0 o3 o6 x0 x3 x6 o0 o3 o6
line o1 o4 o7 x1 x4 x7 o1 o4 o7
line o2 o5 o8 x2 x5 x8 o2 o5 o8
line o0 o4 o8 x0 x4 x8 o0 o4 o8
line o2 o4 o6 x2 x4 x6 o2 o4 o6
line x0 x1 x2 x0 x1 x2 o0 o1 o2
addn 2 res
win
line x3 x4 x5 x3 x4 x5 o3 o4 o5
line x6 x7 x8 x6 x7 x8 o6 o7 o8
line x0 x3 x6 x0 x3 x6 o0 o3 o6
line x1 x4 x7 x1 x4 x7 o1 o4 o7
line x2 x5 x8 x2 x5 x8 o2 o5 o8
line x0 x4 x8 x0 x4 x8 o0 o4 o8
line x2 x4 x6 x2 x4 x6 o2 o4 o6
take1 x4 o4
fl1 o4
fl1 x0
fl1 x8
no
diag
fl1 x2
fl1 x6
forks
fl1 x5
fl1 x7
take1 x8 o8
fl1 x5
fl1 x6
take1 x8 o8
fl1 x7
fl1 x0
take1 x6 o6
fl1 x7
fl1 x2
take1 x8 o8
corner
take1 x0 o0
take1 x2 o2
take1 x6 o6
take1 x8 o8
sides
take1 x1 o1
take1 x3 o3
take1 x5 o5
take1 x7 o7
full
addn 1 res
moved

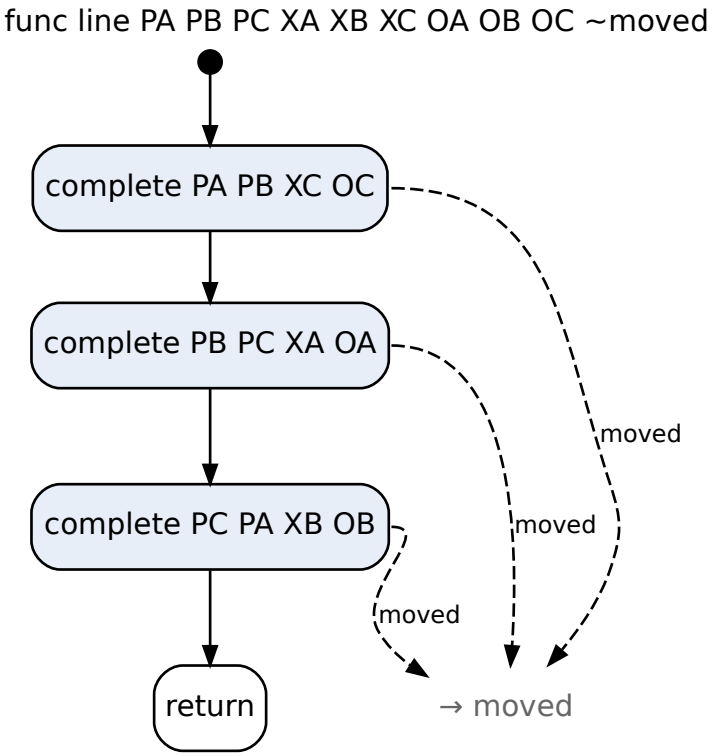


Figure 5: `func line`: complete line ABC whichever square is open.

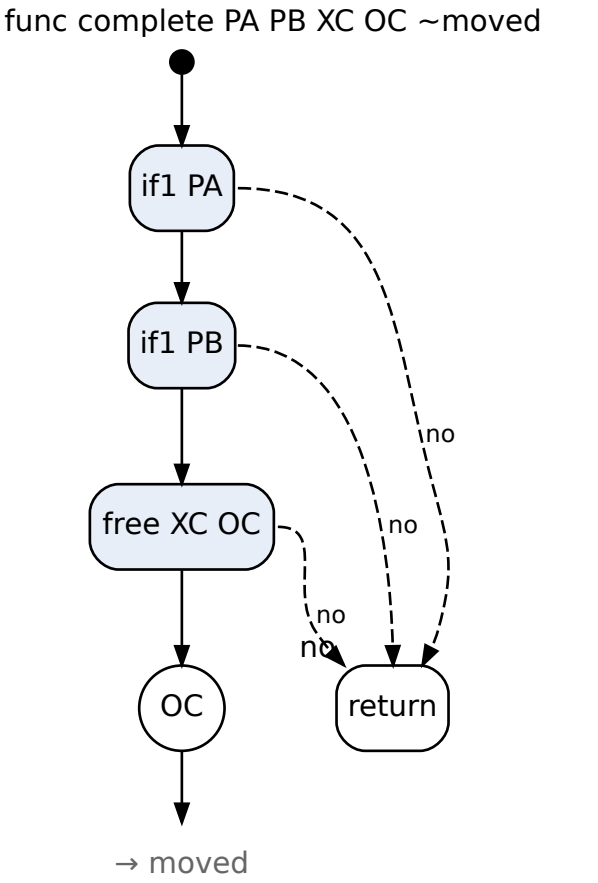


Figure 6: `func complete`: if `PA` and `PB` are held and C is empty, take C.

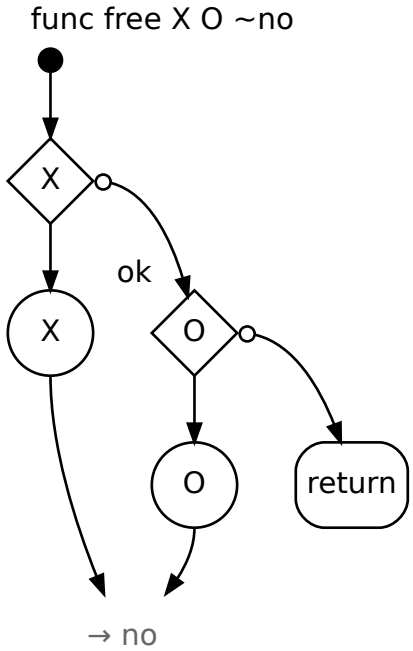


Figure 7: `func free`: fall through if a square is empty.

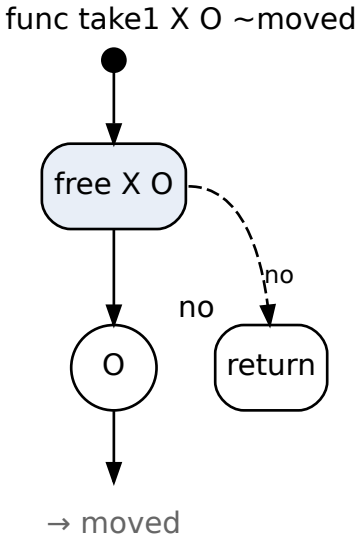


Figure 8: `func take1`: take a square if it is free.

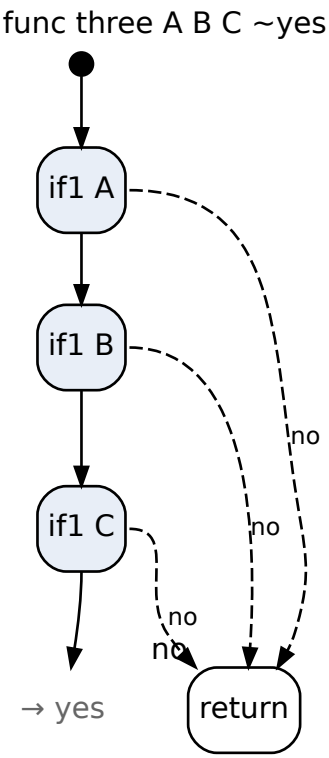


Figure 9: `func three`: are A, B, C all held?

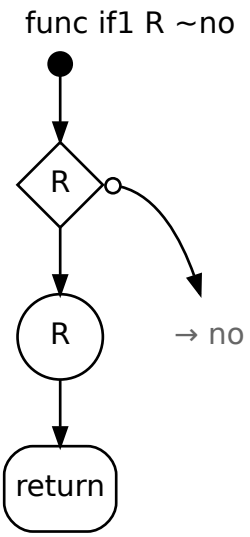


Figure 10: `func if1`: fall through if R holds a bean.

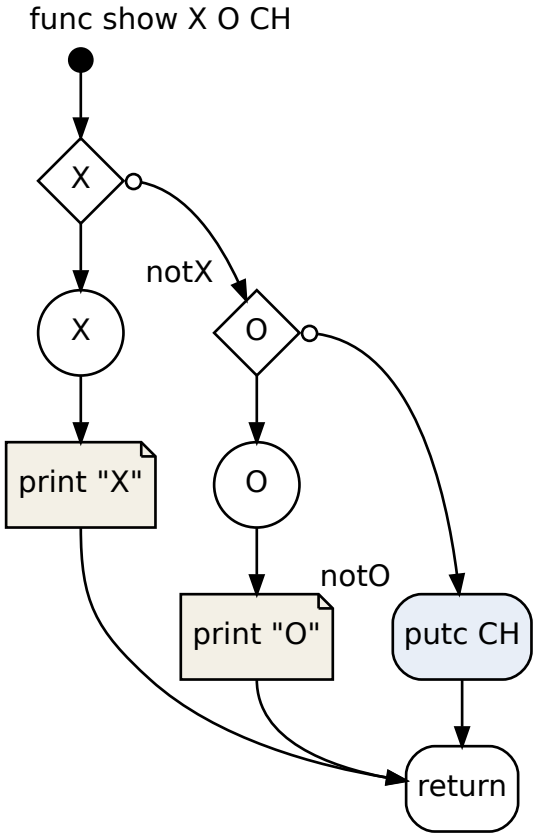


Figure 11: `func show`: print X, O, or the digit.

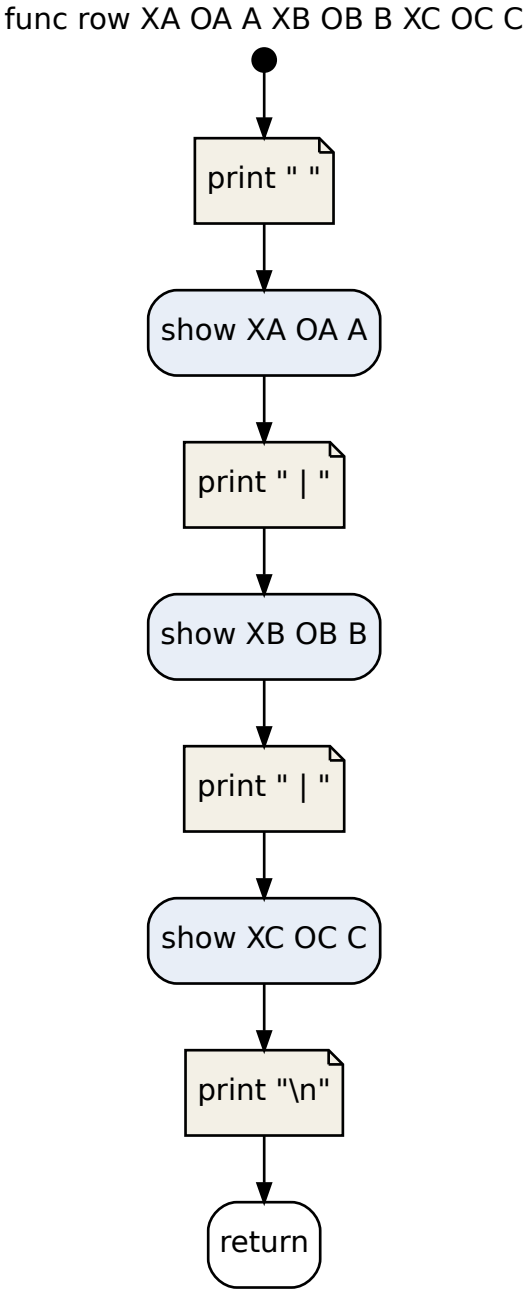


Figure 12: `func row`: print one row of the board.

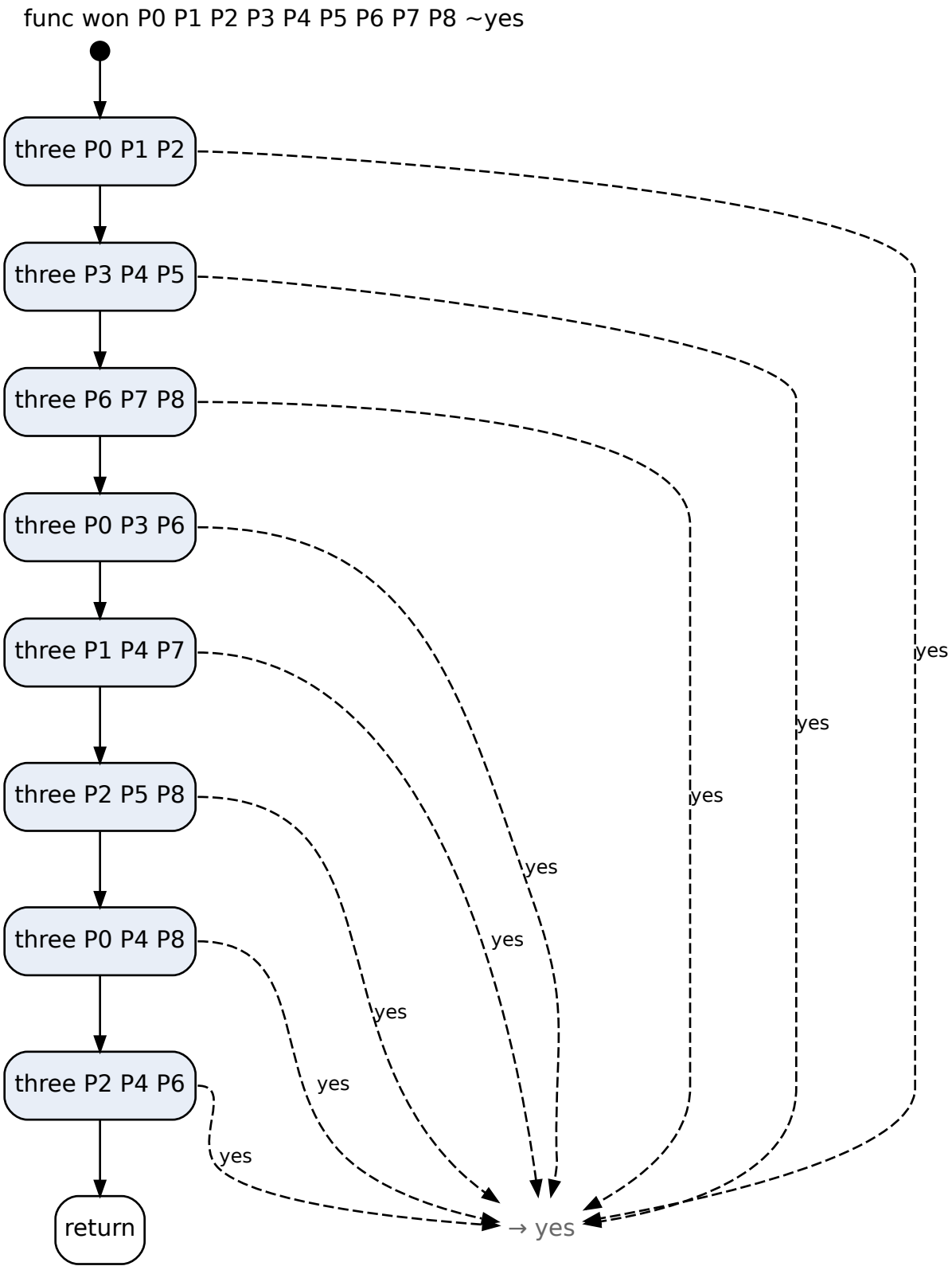


Figure 13: `func won`: has P completed any of the eight lines?

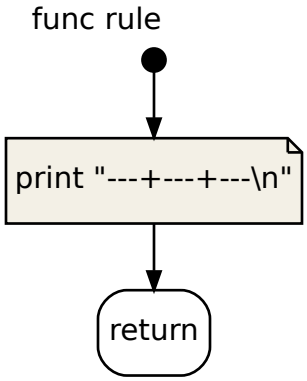


Figure 14: `func rule`: print `---+---+---`.

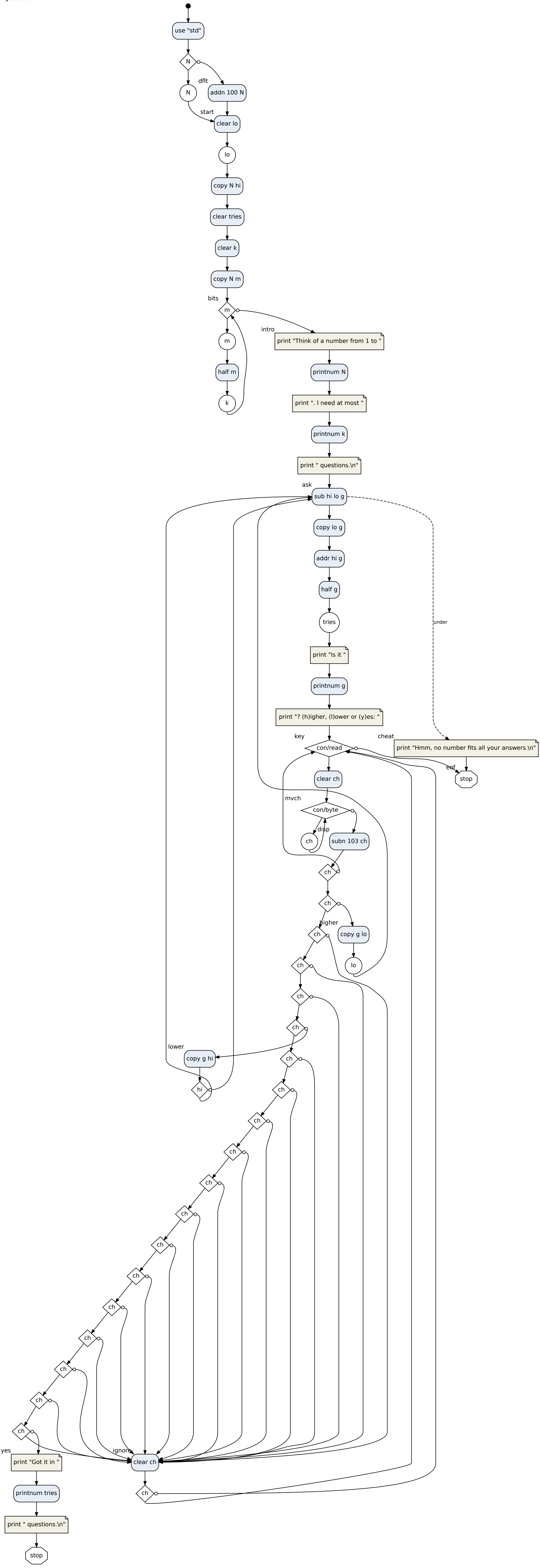
`guess.bc`: binary search

`examples/guess.bc` — 212 source lines · 15495 counter-machine instructions · 273 bins

Think of a number from 1 to N (default 100). The machine keeps the range $[lo, hi]$ that your number must lie in and always asks about the middle, so each answer halves the range: at most $\lceil \log_2(N+1) \rceil$ questions, which it works out and tells you first. If the range ever empties, your answers contradicted each other, and it says so. With `-O`, $N = 10^{30}$ takes at most 100 questions.

```
./beancraft examples/guess.bc                    answer h, l or y
```

guess.bc


use "std"
N
dflt
N
addn 100 N
start
clear lo
lo
copy N hi
clear tries
clear k
copy N m
bits
m
m
half m
k
intro
print "Think of a number from 1 to "
printnum N
print ". I need at most "
printnum k
print " questions.\n"
ask
sub hi lo g
copy lo g
addr hi g
half g
tries
print "Is it "
printnum g
print "? (h)igher, (l)ower or (y)es: "
under
cheat
print "Hmm, no number fits all your answers.\n"
key
con/read
eof
stop
clear ch
mvch
con/byte
disp
ch
subn 103 ch
ch
ch
higher
ch
copy g lo
lo
ch
ch
ch
ch
lower
copy g hi
hi
ch
ch
ch
ch
ch
ch
ch
ch
ch
ch
ch
ch
ch
yes
print "Got it in "
printnum tries
print " questions.\n"
stop
ignore
clear ch
ch

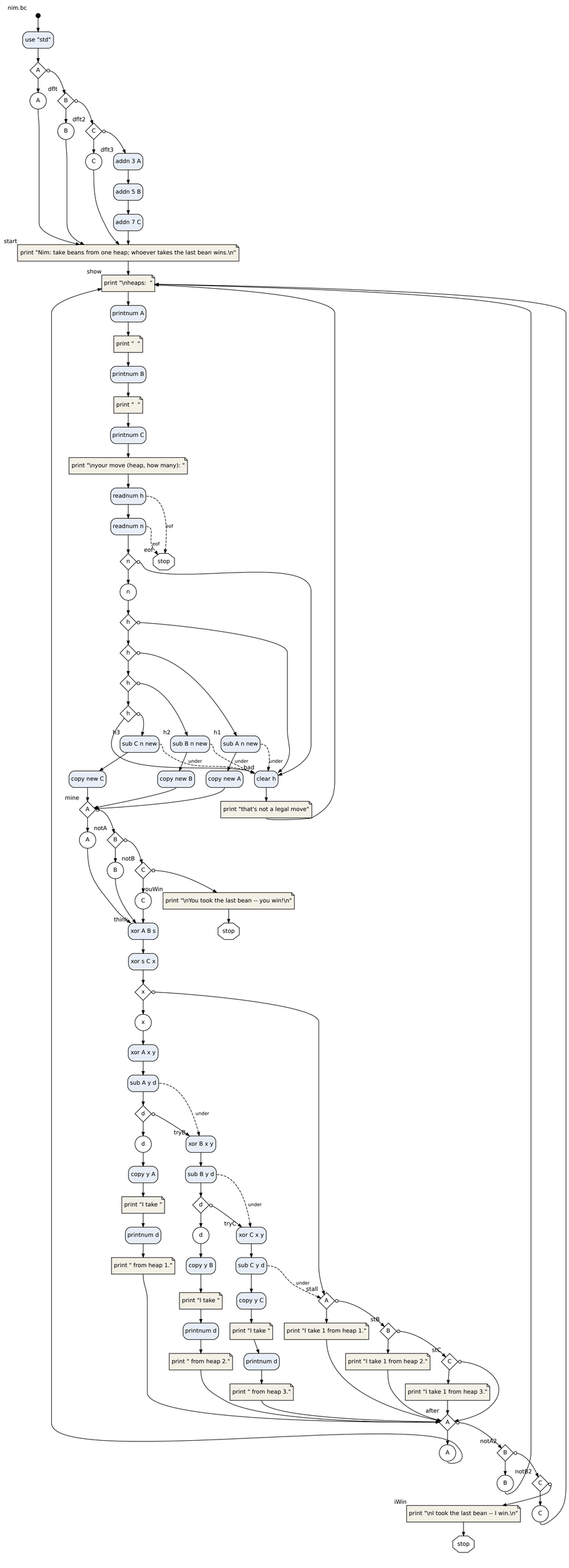
`nim.bc`: perfect play from a theorem

`examples/nim.bc` — 426 source lines · 32058 counter-machine instructions · 697 bins

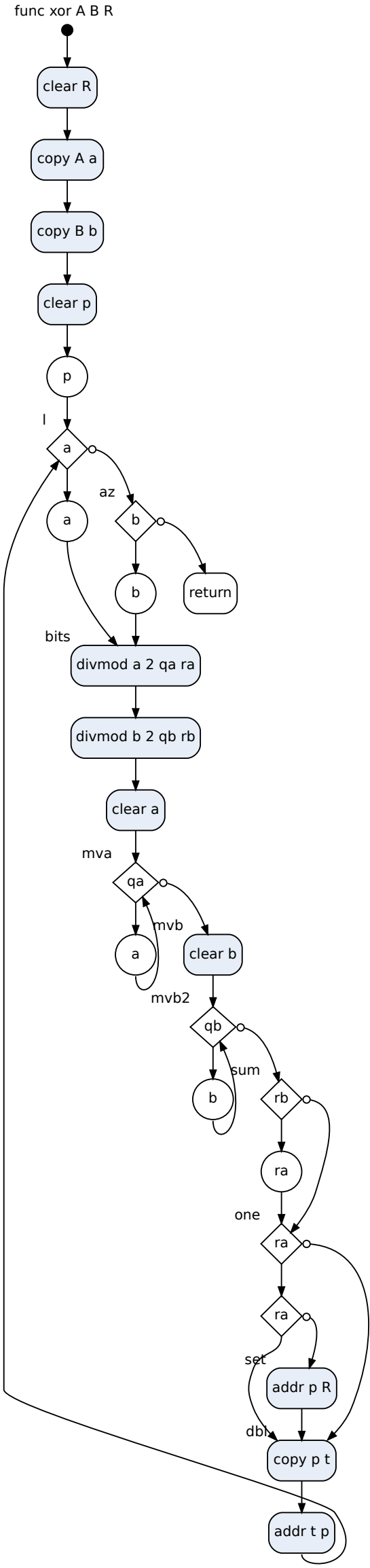
Three heaps (default 3, 5, 7); take any number from one heap; taking the last bean wins. Bouton (1901): XOR the heap sizes in binary. If that *nim-sum* is 0, the player to move loses against good play; otherwise shrinking some heap h to $h \oplus s$ makes it 0. The machine never looks ahead. `func xor` peels off one bit of each number per round with `divmod a 2`, and `sub` (which jumps to `under` when the result would be negative) both finds the heap to shrink and says how much to take.

In 40 games from random starting heaps, the machine won every one in which the human started from a nim-sum of 0. From 3, 5, 7 a perfect human wins.

```
./beancraft examples/nim.bc                                   type e.g. 3 2: take 2 from heap 3
./beancraft -O examples/nim.bc A=10000000000000000000 B=12345678987654321 C=3
```


nim.bc
use "std"
A
A
dflt
B
B
dflt2
C
C
dflt3
addn 3 A
addn 5 B
addn 7 C
start
print "Nim: take beans from one heap; whoever takes the last bean wins.\n"
show
print "\nheaps: "
printnum A
print " "
printnum B
print " "
printnum C
print "\nyour move (heap, how many): "
readnum h
readnum n
eof
eof
stop
n
n
h
h
h
h
h3
h2
h1
sub C n new
sub B n new
sub A n new
under
under
under
<0 : bad
copy new C
copy new B
copy new A
clear h
print "that's not a legal move"
mine
A
notA
A
B
notB
B
C
youWin
C
print "\nYou took the last bean -- you win!\n"
stop
think
xor A B s
xor s C x
x
x
xor A x y
sub A y d
d
under
tryB
d
xor B x y
copy y A
sub B y d
print "I take "
d
under
tryC
printnum d
d
xor C x y
print " from heap 1."
copy y B
sub C y d
under
stall
print "I take "
copy y C
A
printnum d
print "I take "
print "I take 1 from heap 1."
stB
print " from heap 2."
printnum d
B
print "I take 1 from heap 2."
stC
print " from heap 3."
C
print "I take 1 from heap 3."
after
A
A
notA2
B
B
notB2
C
C
iWin
print "\nI took the last bean -- I win.\n"
stop

`func xor A B R`: $R = A \oplus B$, one bit per round.



`wythoff.bc`: the golden ratio, from beans

`examples/wythoff.bc` — 396 source lines · 29462 counter-machine instructions · 743 bins

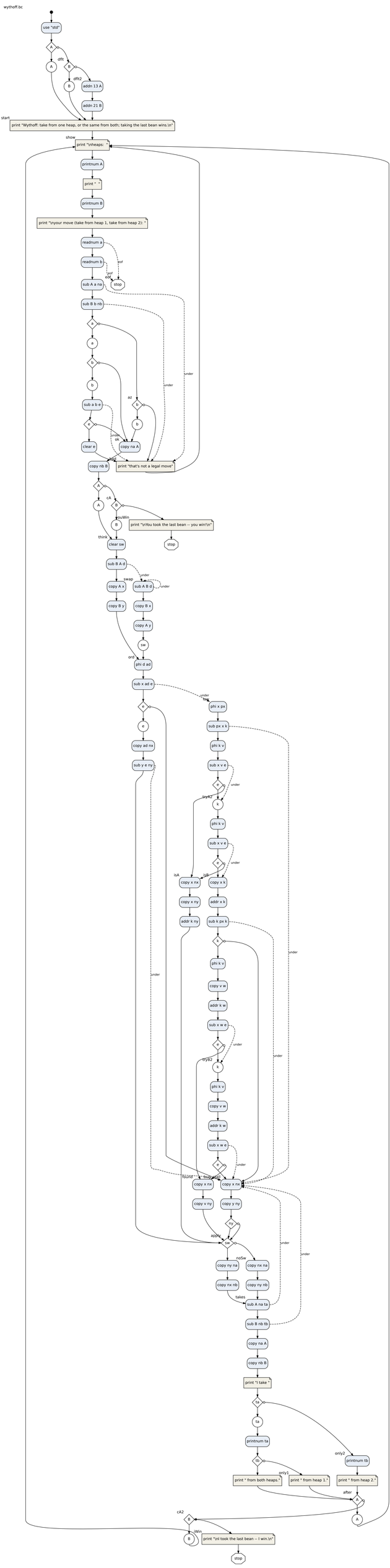
Two heaps (default 13, 21); take any number from one heap, or the same number from both. Wythoff (1907) showed that the losing positions are the pairs $(a_k, a_k + k)$ with $a_k = \lfloor k\varphi \rfloor$, $\varphi = (1 + \texttt{sqrt}\ 5)/2$: (0,0), (1,2), (3,5), (4,7), (6,10), (8,13), … From any other position there is a move onto one, found by comparing the smaller heap with a_d where d is the difference. The machine has no φ, and needs none:

$$\lfloor k\varphi \rfloor = \left(k + \left\lfloor \sqrt{5k^2} \right\rfloor\right) \div 2$$

exactly, so `func phi` computes the golden ratio, to just the precision needed, from integer square roots of beans.

Checked against a brute-force solution for every position up to 60×60, and in about 60 played games every machine reply landed on a losing pair.

```
./beancraft examples/wythoff.bc                    type a b: take a from heap 1, b from heap 2
```


wythoff.bc
use "std"
A
dflt
B
A
dflt2
addn 13 A
B
addn 21 B
start
print "Wythoff: take from one heap, or the same from both; taking the last bean wins.\n"
show
print "\nheaps: "
printnum A
print ", "
printnum B
print "\nyour move (take from heap 1, take from heap 2): "
readnum a
readnum b
eof
eof
not
stop
sub A a na
sub B b nb
a
a
b
b
sub a b e
e
under
ok
clear e
copy na A
copy nb B
print "that's not a legal move"
under
under
ad
B
b
A
cA
B
youWin
B
print "\nYou took the last bean -- you win!\n"
think
clear sw
stop
sub B A d
under
swap
copy A x
sub A B d
under
copy B y
copy B x
copy A y
sw
ord
phi d ad
sub x ad e
under
hop
e
e
copy ad nx
sub y e ny
phi x px
sub px x k
phi k v
sub x v e
e
under
tryA2
k
phi k v
sub x v e
e
under
isA
isB
copy x nx
copy x k
copy x ny
addr x k
addr k ny
sub k px k
k
phi k v
copy v w
addr k w
sub x w e
e
under
tryB2
k
phi k v
copy v w
addr k w
sub x w e
e
under
found
backOn
copy x nx
copy x nx
copy v ny
copy y ny
ny
applyy
ex
under
under
noSw
copy ny na
copy nx na
copy nx nb
copy ny nb
takes
sub A na ta
sub B nb tb
copy na A
copy nb B
print "I take "
ta
ta
printnum ta
tb
only2
printnum tb
only1
print " from both heaps."
print " from heap 1."
print " from heap 2."
after
A
A
cA2
B
youWin
B
print "\nI took the last bean -- I win.\n"
stop

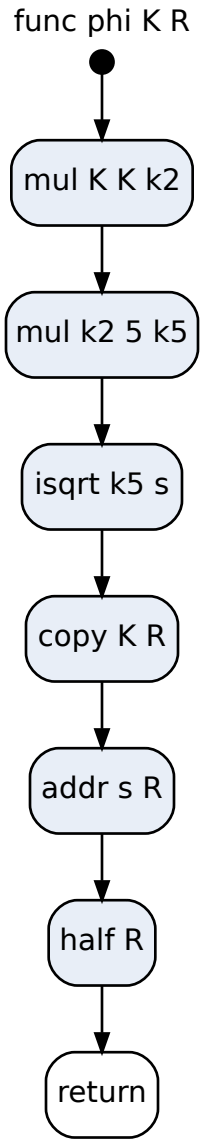


Figure 15: `func phi K R`: $R = \lfloor K\varphi \rfloor$.

`hexapawn.bc`: a machine that learns

`examples/hexapawn.bc` — 2399 source lines · 30261 counter-machine instructions · 1034 bins

The others play by rules someone worked out. This one is told nothing about strategy. Hexapawn is played with three pawns each on a 3×3 board; a pawn steps forward onto an empty square or diagonally onto an enemy pawn; you win by reaching the far row, taking every enemy pawn, or leaving the other side with no move. You are White and move first.

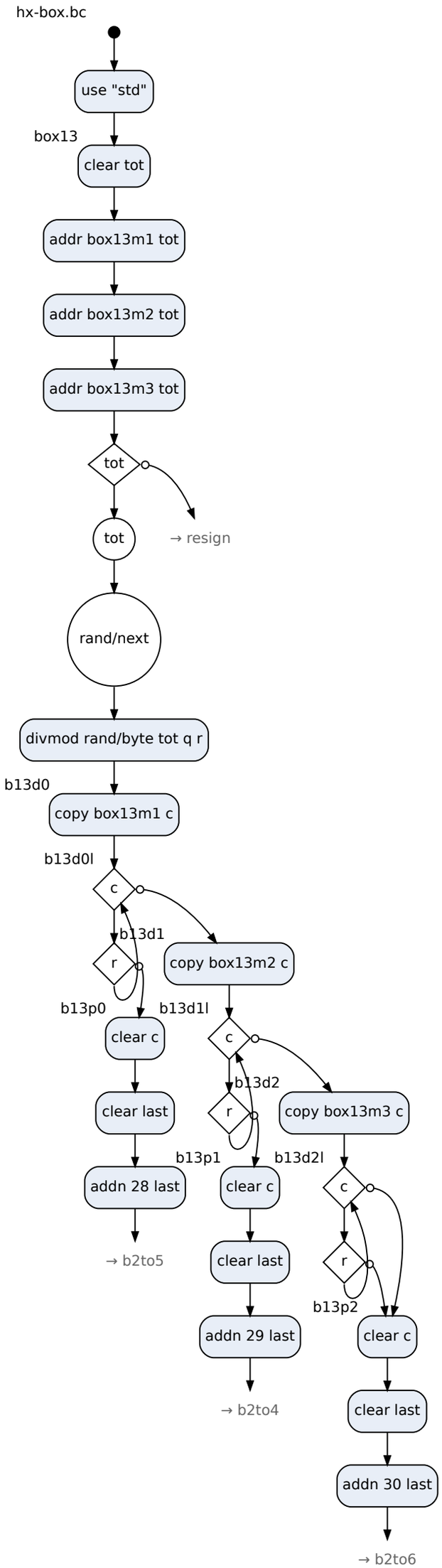
The machine is Martin Gardner's *Hexapawn Educable Robot* (1962), after Donald Michie's MENACE (1961), which learned noughts and crosses from matchboxes full of coloured beads. There is a matchbox for each of the 37 positions the machine can face, and in each box one bin per legal move (90 bins), each starting with one bean. To move, it draws a bean at random from the box for the current position. When it loses, the bean it played last is removed, so that move is never made again; if a box runs empty it resigns, and the move that led there loses its bean in turn. Losses push the blame backwards, game by game.

With `T=n` it first plays n practice games against a random opponent, printing W or L for each. Measured from the program's own bins over ten runs each: after 300 practice games, 5 of 10 runs could still be beaten by some line of play; after 1000, none could. It keeps learning from your games too.

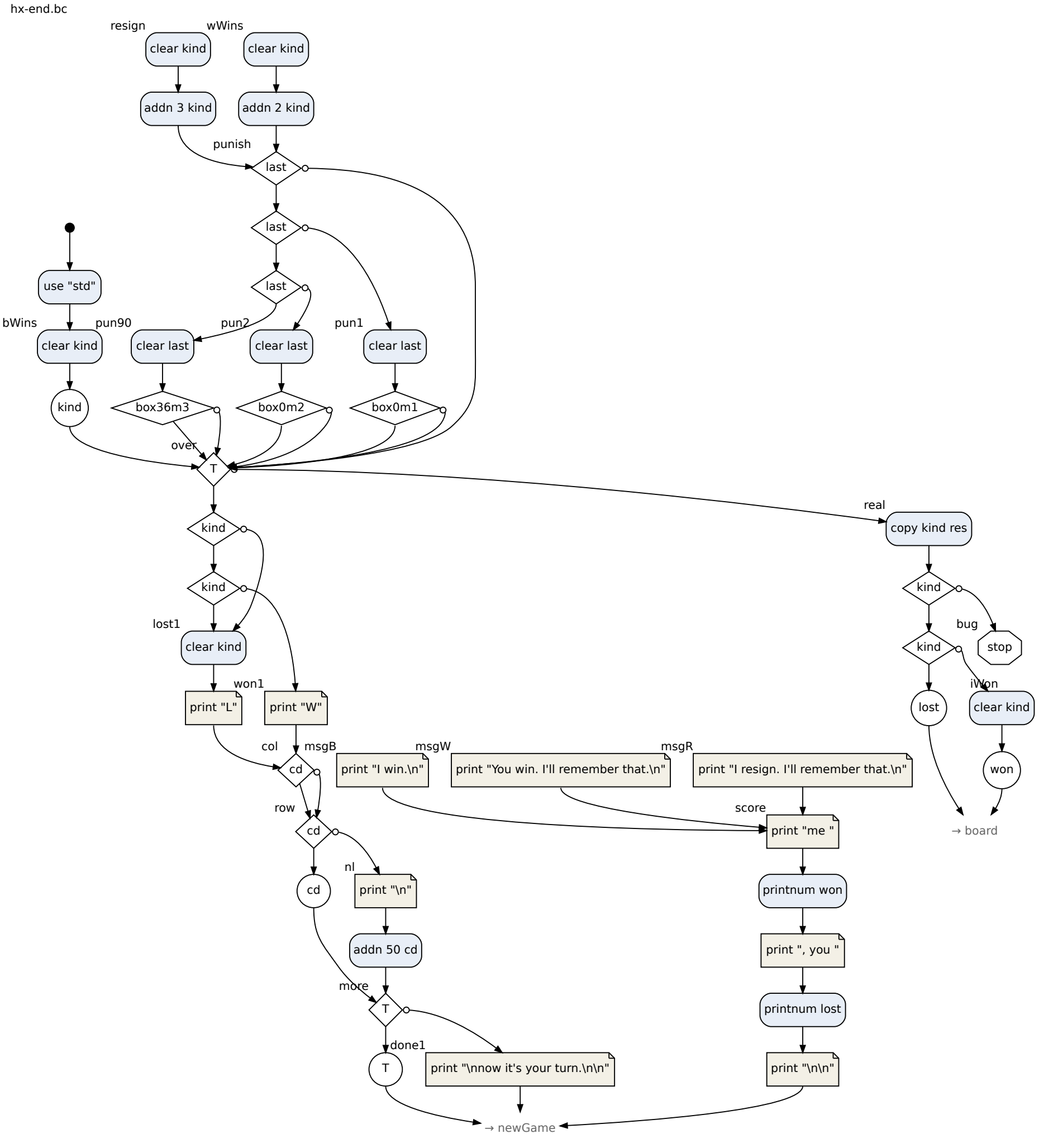
```
./beancraft examples/hexapawn.bc T=1000
practice (W: B won, L: B lost):
LLLWWWWLLLWWLLWLWWWWWLWWWLWLLWWWWWWLLWLWWWWWWWWWW
LWWWWWWWWLWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWW …
```

The program is about 2,500 lines, generated by `scripts/gen_hexapawn.py` (the move rules and the position recognition; the learning is all in the bins), so its whole graph is too big to draw. Three excerpts:

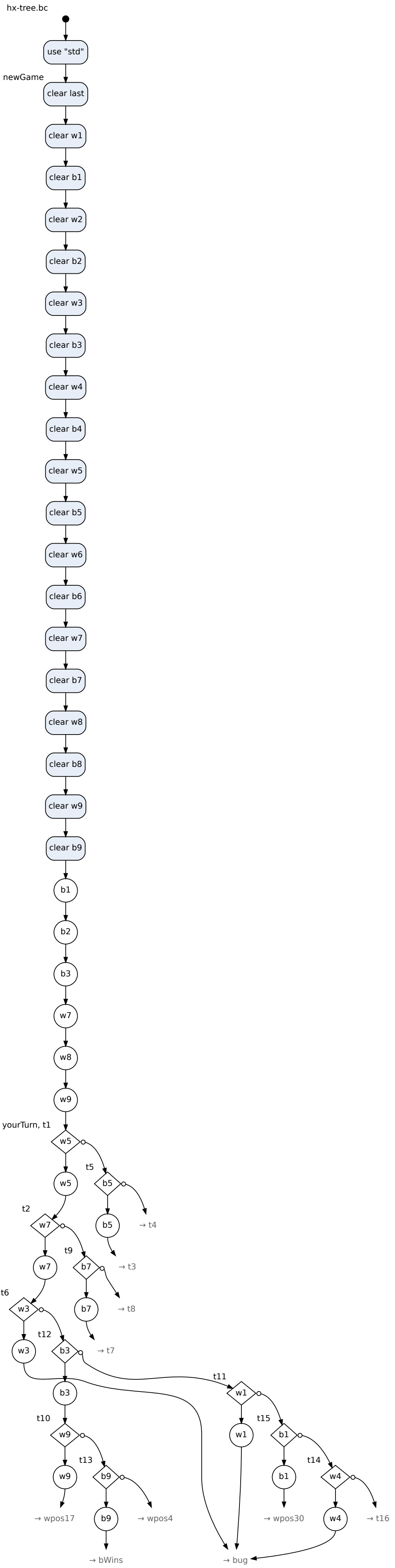
One matchbox, box 13 (board `.BB / W.W / W..`, three moves). It adds up the beans, resigns if there are none, draws a random number `r` below the total, and walks the bins until `r` runs out inside one of them. That bin's number is remembered in `last`.



The end of a game. On a loss, `punish` finds bin number `last` with a chain of `take`s (one per bin; most of the chain is cut from this picture) and removes one bean from it. A practice game then prints W or L; a real one prints the board, the verdict and the score.



A new game, and the start of the decision tree that recognises the position: one square at a time, it asks whether that square's W bin or B bin holds a bean, and branches three ways.



The standard library

The `funcs` from `examples/std.bc` that these programs use. `mul`, `divmod` and `sub` are written in exactly the shapes the `-O` optimizer folds into single big-number operations.

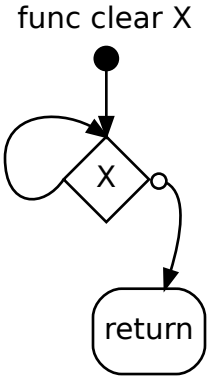


Figure 16: `clear X`: $X \leftarrow 0$.

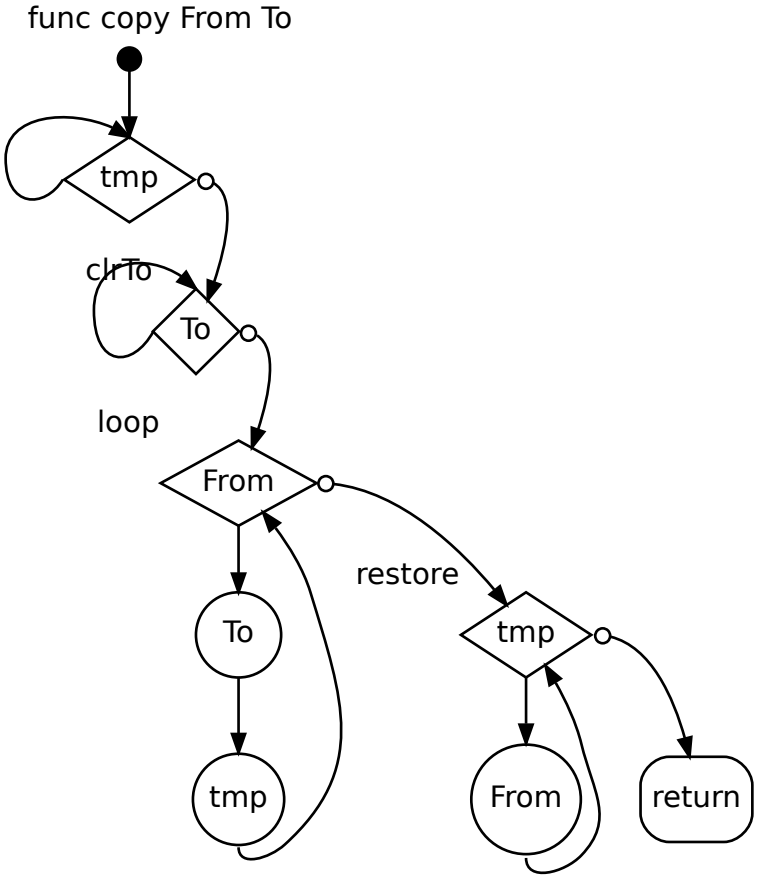


Figure 17: `copy From To`: $\mathrm{To} \leftarrow \mathrm{From}$ (via a scratch bin).

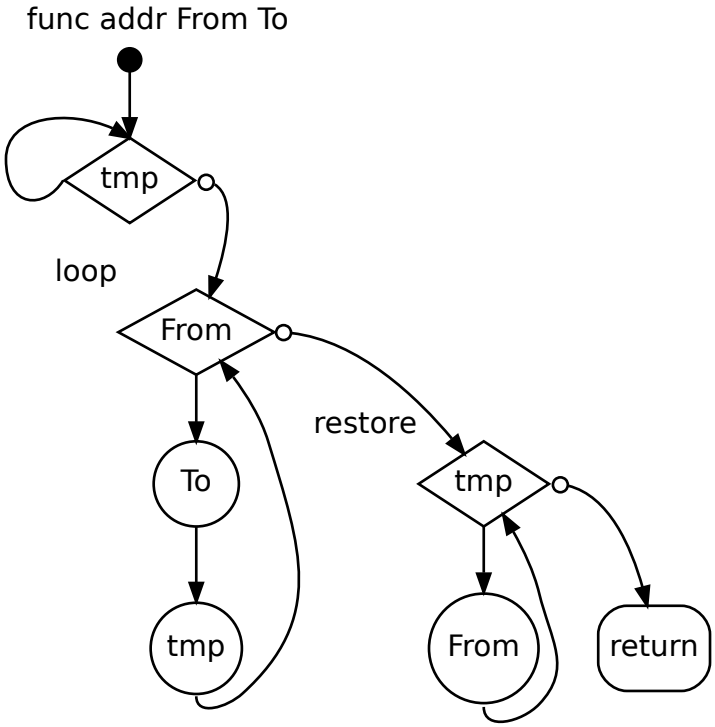


Figure 18: `addr From To`: $\mathrm{To} \mathrel{+}= \mathrm{From}$.

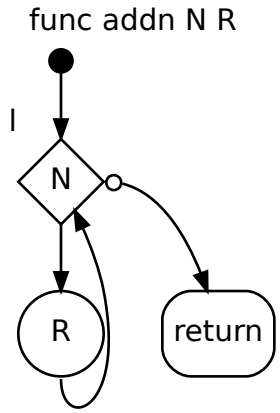


Figure 19: `addn N R`: $R \mathrel{+}= N$ (N a literal, usually).

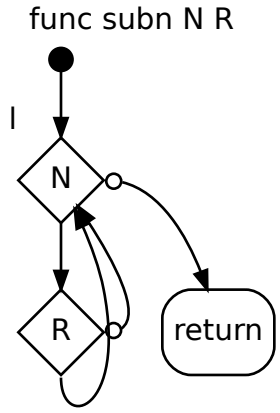


Figure 20: `subn N R`: $R \leftarrow \max(0, R - N)$.

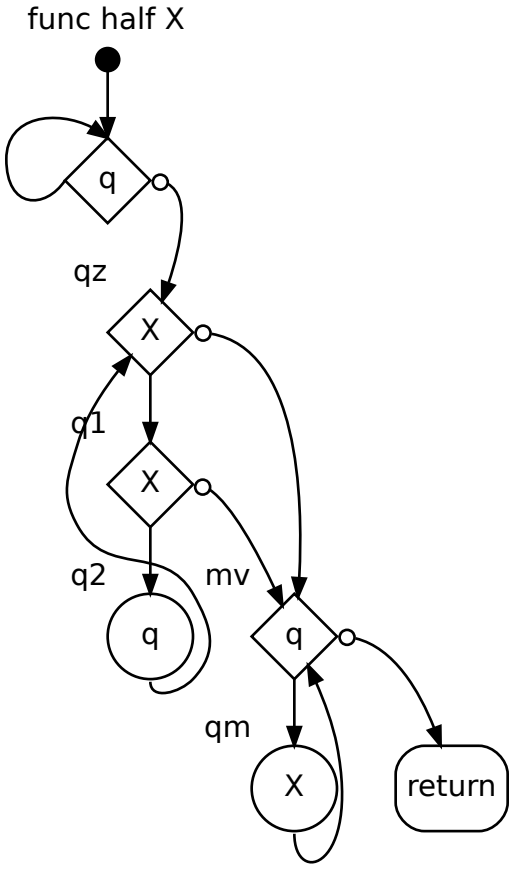


Figure 21: `half X`: $X \leftarrow \lfloor X/2 \rfloor$.

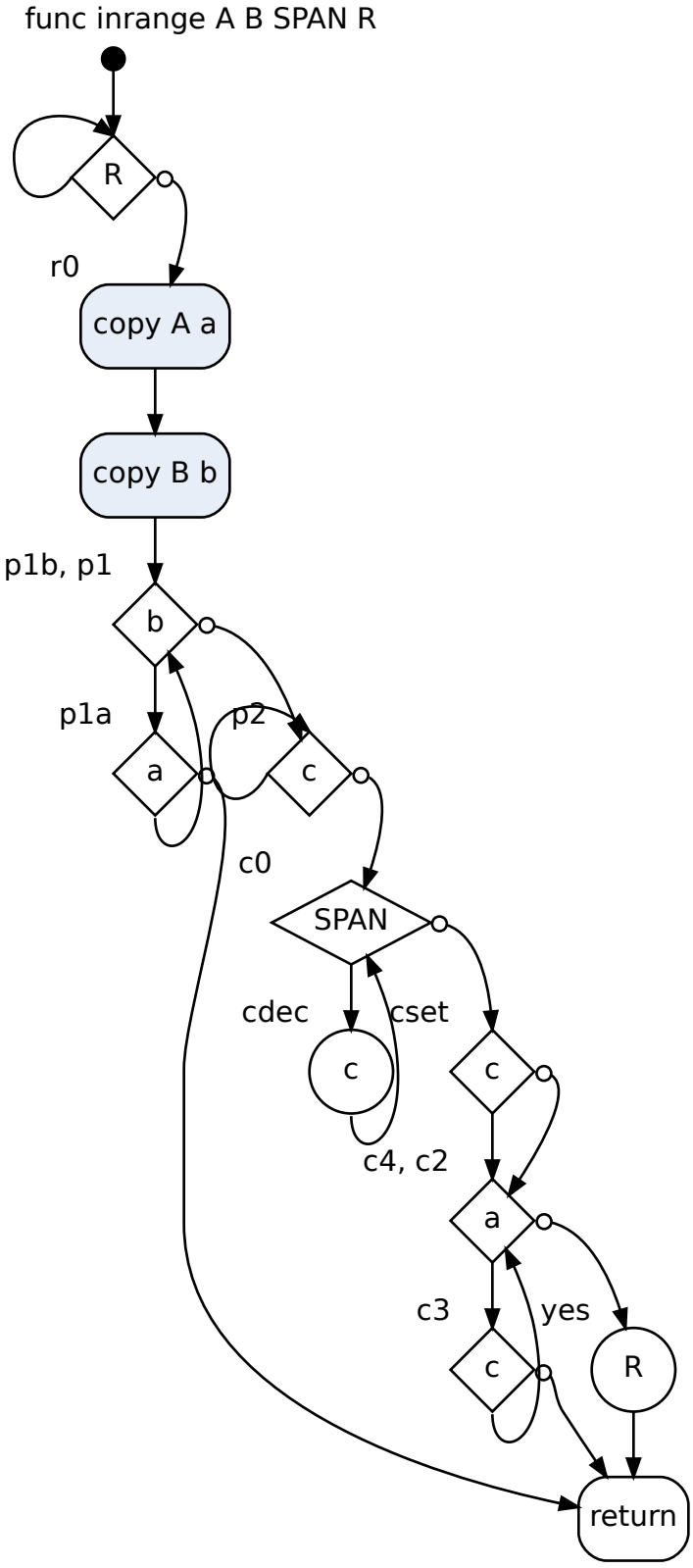


Figure 22: `inrange A B SPAN R`: is $B \leq A < B + \mathrm{SPAN}$?

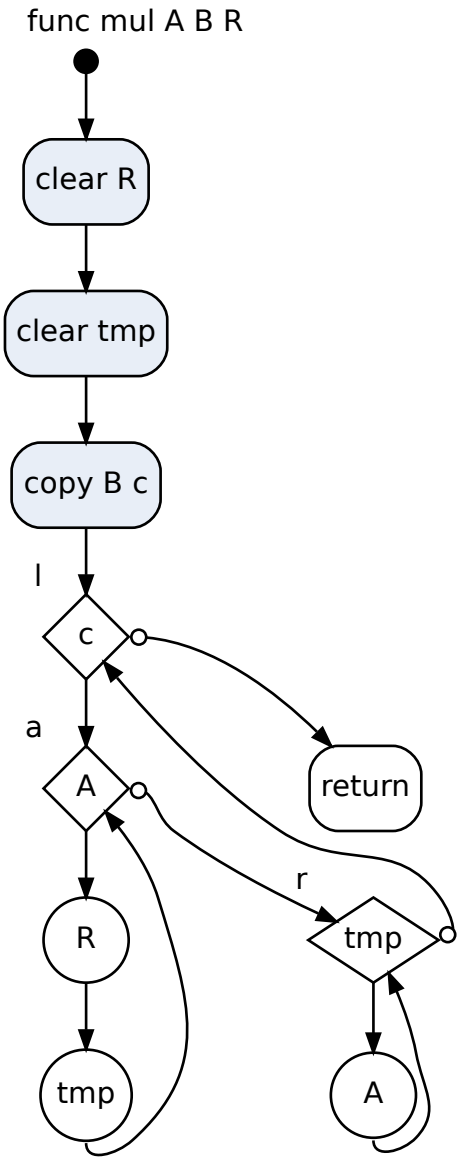


Figure 23: `mul A B R`: $R \leftarrow AB$.

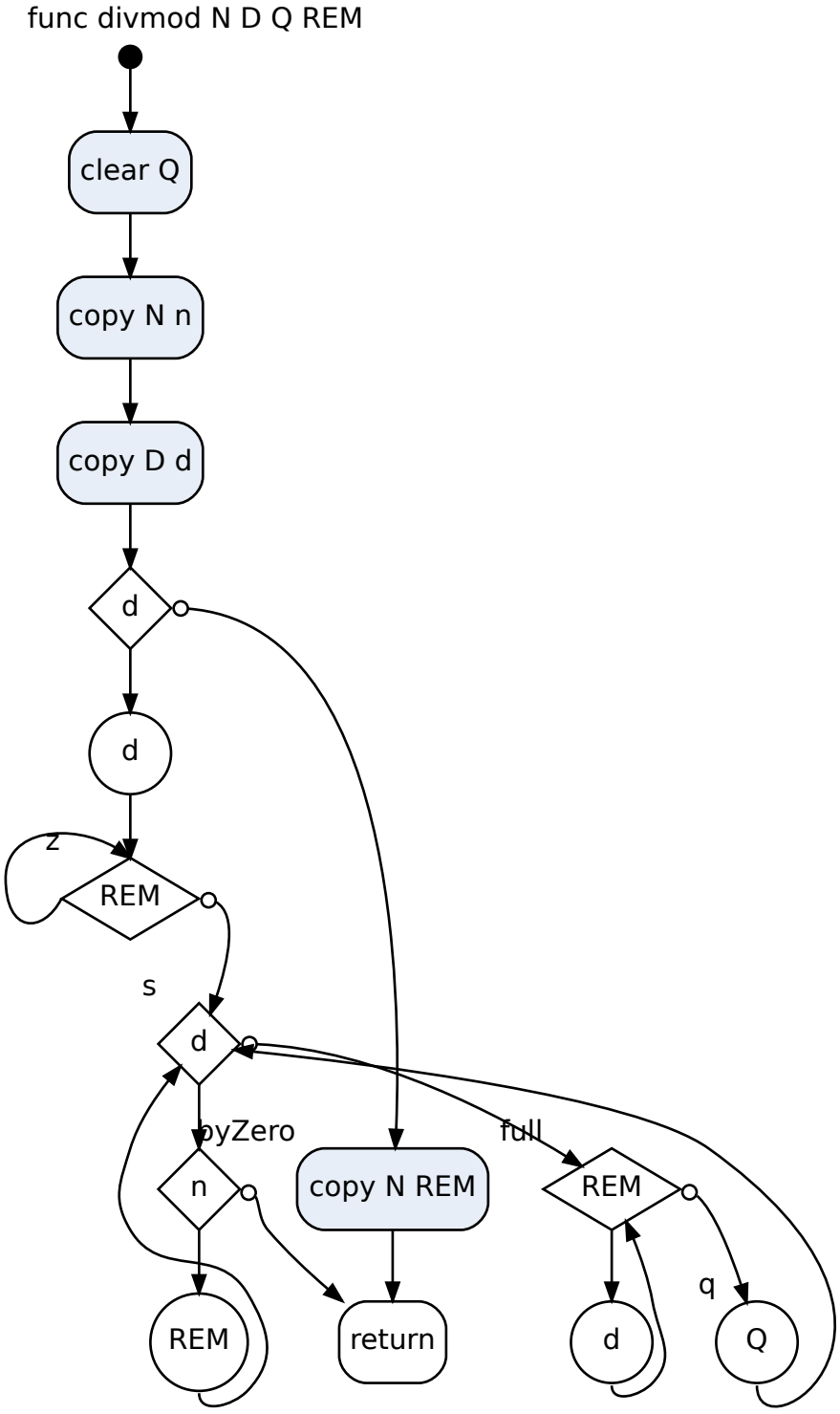


Figure 24: `divmod N D Q REM`: long division.

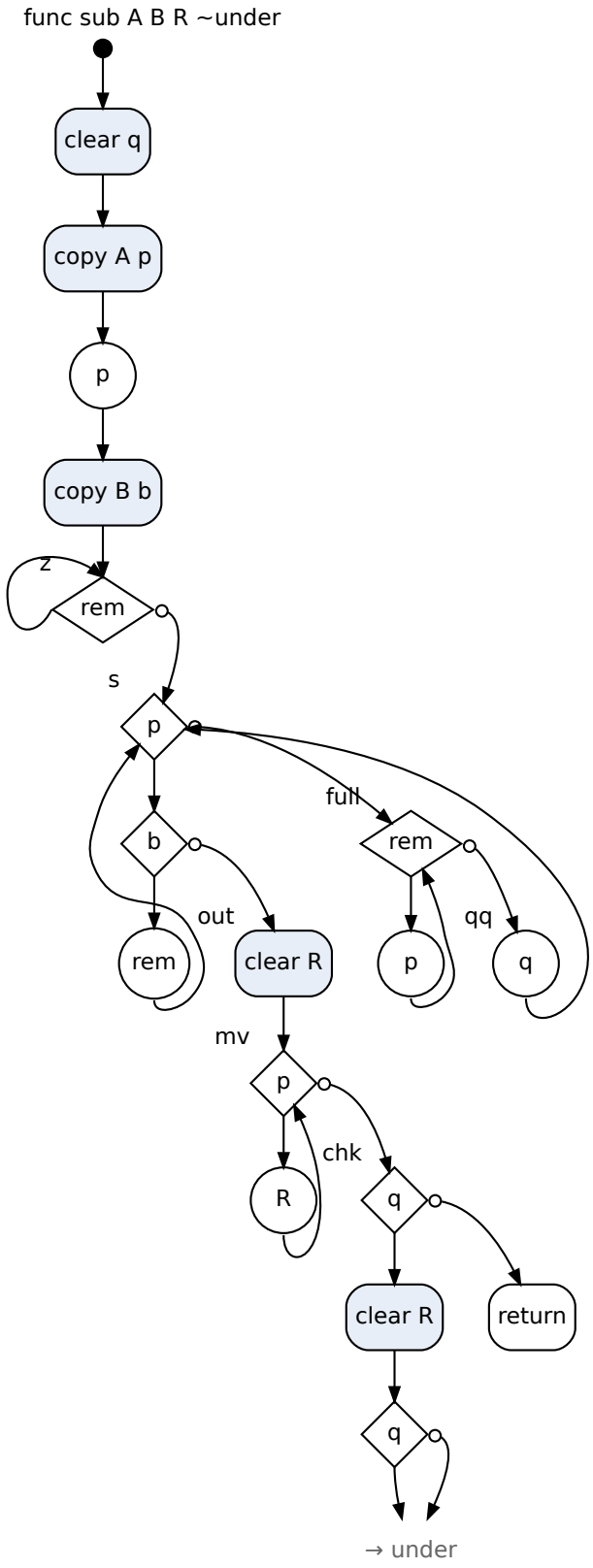


Figure 25: `sub A B R ~under`: $R \leftarrow A - B$, or jump to `under`.

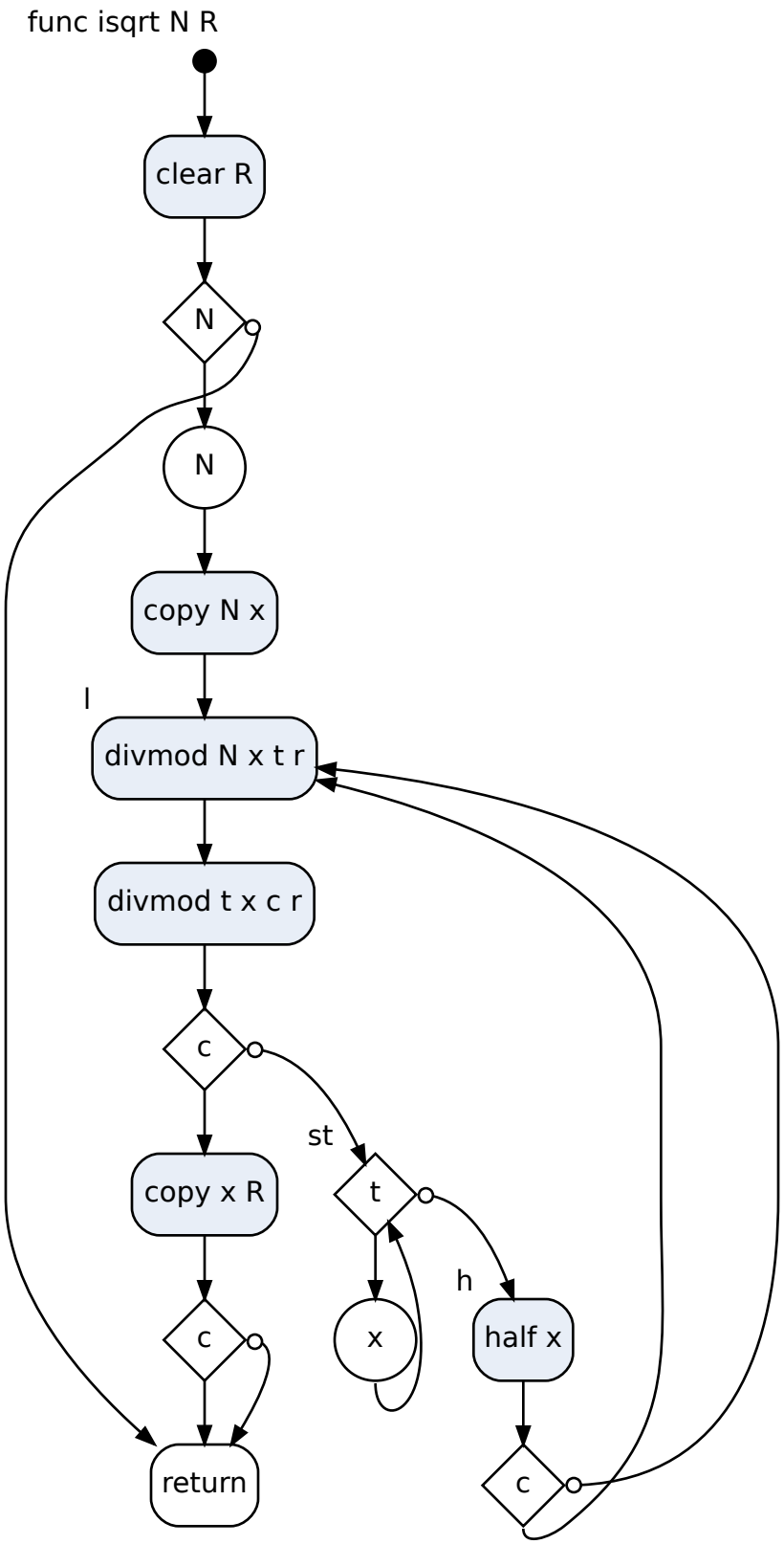
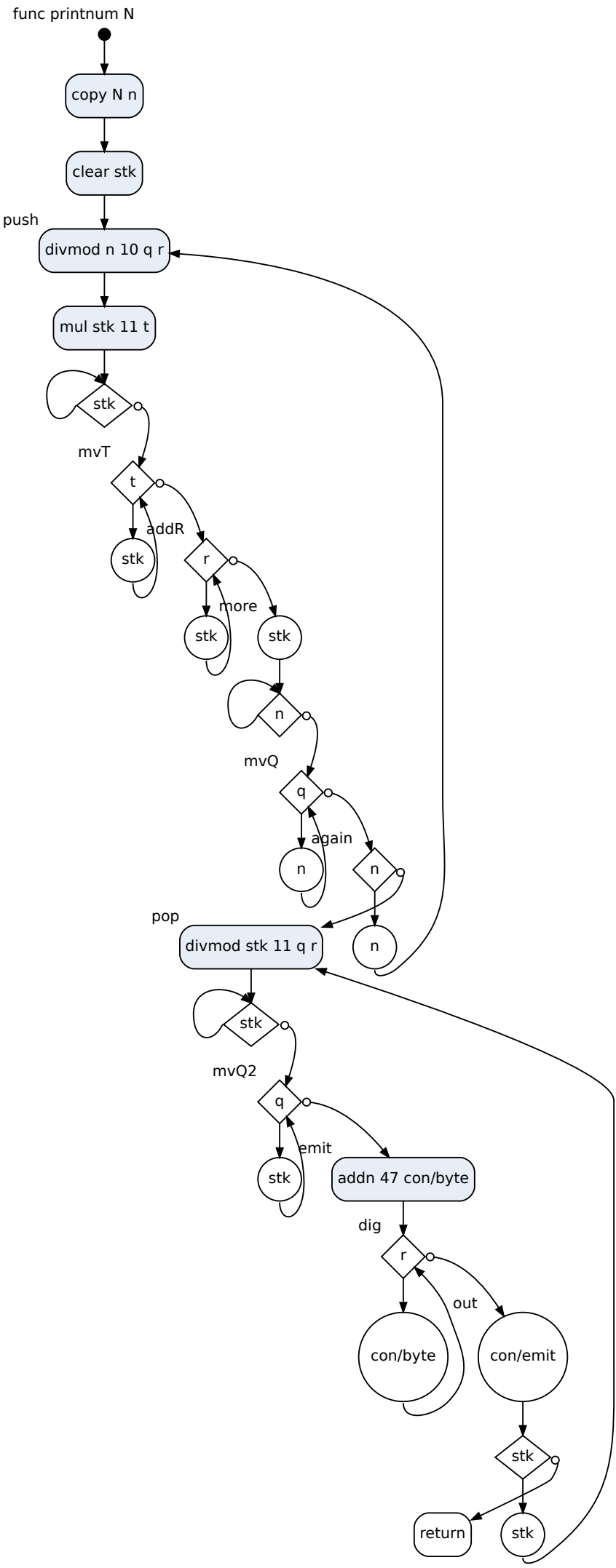


Figure 26: `isqrt N R`: Newton's method, as in `isqrt.bc`.

`printnum N`: print N in decimal (digits stacked in base 11).



`readnum N ~eof`: read a decimal number from the console.

